그래픽스 학습·구현과 아카이브 자동화

DirectX11 기반 그래픽스 학습 코드와 일부 개인 확장 구현을 정리한 프로젝트입니다.
분산된 학습 기록을 코드 설명·실행 결과·검증 자료로 구성하고, 반복 정리를 Codex로 자동화했습니다.

반복 작업의 규격화

학습 내용과 구현 결과를 설명할 수 있도록,
반복되는 문서·실행·기록 절차를 규격화했습니다.

공통 규격을 적용한 README 구성 예시

02_Bloom

원문 발췌 · 중간 생략

구현 요약

입력 이미지를 CPU에서 처리한 뒤 dynamic texture로
업로드하고 full-screen quad로 표시한다.

Build And Run

항목	결과
Debug x64 build/run	성공
Release x64 build/run	성공

Related Docs

Topic ↗
Verification ↗
Detailed Demo ↗

공통 README 템플릿 ↗ Bloom README 원문 ↗

실제 산출물 · 시각 결과 기록



Bloom 설명·결과 이미지 ↗

작업 기여 | **사용자 작업:** 정리 목적·기준 제시, 결과 검토 | **Codex 작업:** 코드·기록 조사, 문서 초안 작성, 실행 결과 수집

코드·문서·실행 결과 색인

왼쪽의 주제·페이지 링크는 본문으로, 오른쪽의 Git 링크는 관련 원문으로 이동합니다.
그래픽스 학습·구현 01~13은 상세 페이지의 번호와 같습니다.

AI 아카이브 자동화

자동화 과정 (3페이지) → Git 문서 ↗
문제 해결 01 (4–6페이지) → Git 문서 ↗

학습 범위·작업 기록

학습·구현 범위 (10페이지) → Git 저장소 ↗

렌더링 기초·표면 표현

01 반사·굴절 (11페이지) → Git 문서 ↗
02 원근 투영·UV 보정 (12페이지) → Git 문서 ↗
03 DirectX11 파이프라인 (13페이지) → Git 문서 ↗
04 Normal Mapping (14페이지) → Git 문서 ↗
05 Height Mapping (15페이지) → Git 문서 ↗
06 Tessellation (16페이지) → Git 문서 ↗

FBX 구조 호환 문제

문제 해결 02 (7–9페이지) → Git 코드 ↗

재질·후처리·입력·애니메이션

07 PBR·IBL (17페이지) → Git 문서 ↗
08 HDR·Bloom (18페이지) → Git 문서 ↗
09 3D 선택·조작 (19페이지) → Git 문서 ↗
10 스켈레탈 애니메이션 (20페이지) → Git 문서 ↗

GPU 연산·인스턴싱

11 GPU Blur (21페이지) → Git 문서 ↗
12 GPU 연기 (22페이지) → Git 문서 ↗
13 잔디 인스턴싱 (23페이지) → Git 문서 ↗

링크 안내 본문 보기: 포트폴리오 내부 이동 / Git 문서·코드: 외부 근거

AI를 활용한 아카이브 구축 자동화

대표 예제로 문서 규격과 작업 절차를 검토한 뒤,
Codex의 작업 범위를 후속 예제의 문서 정리·실행·촬영·검사로 넓혔습니다.

자동화 진행 과정과 작업 기여

01 정리 기준 설정

사용자 작업

문서·실행 결과의 규격 제시
촬영 방법·조건·제한사항을
정리해 전달

Codex 작업

자료 조사·문서 초안 작성
요구에 맞춘 촬영 도구 구현

→

02 대표 예제 적용·보완

사용자 작업

대표 예제의 결과 검토
필요한 보완 요청

Codex 작업

문서·검증 기록·데모 연결
요청에 따른 작업 절차 보완

→

03 후속 예제로 확대

사용자 작업

진행 상황 관찰
예외 발생 시 방향 조정

Codex 작업

후속 예제 문서 정리
실행·촬영·형식 검사

→

04 산출물 검수

사용자 작업

문서·촬영 결과와
학습 내용 대조
정리 목적에 맞게 보완 요청

Codex 작업

보완 의견에 따른 자료 수정
문서와 근거 링크 정리

오류 발생 시 대응 방식

사용자 작업: 분석 과정 관찰 · 맥락이 어긋나면 방향 조정

Codex 작업: 오류 원인 1차 분석

작업 기록 · 대표 예제 적용 ↗

촬영 도구의 목적

목적: 예제의 시각적 실행 결과를 사진·영상으로 기록

사용 도구: Windows API · FFmpeg

도구 문서 · 실행 화면 촬영 ↗

자동화 범위 | 예제 검토 · 문서 정리 · 실행 결과 기록 · 형식 검사

문제: 실패 상태를 구분하지 못해 자동 촬영 작업 반복

자동 촬영 과정에서 이전 실행의 오류 창이 남아, 새 실행 결과를 판단하는 데 혼선이 생겼습니다.
오류 창 하나를 닫은 뒤 정리 완료를 확인하지 않고 다음 작업을 진행한 것이 원인이었습니다.

재현 시연 · 새 실행 판단에 섞인 이전 오류



비교 재현 영상 ↗

작업 흐름 · 오류 대응 반복

Codex가 잔존 오류를 새 실행 문제와 혼동해,
분석·수정·재실행을 반복했습니다.

관찰 · 이전 오류가 새 실행에 잔존

오류 A를 닫자 후속 오류 B가 나타났지만,
이를 남긴 채 새 실행을 진행했습니다.

원인 · 정리 완료 판단 누락

오류 창 하나를 닫은 뒤
후속 창이 남았는지 확인하지 않고
다음 작업으로 진행했습니다.

오류 처리 기록 ↗

작업 기여 | **사용자 작업:** 오류 대응 과정 관찰·방향 제시 | **Codex 작업:** 당시 오류 분석·수정·재실행 시도

해결: 오류 정리 결과를 판별하도록 자동화 도구 보완

오류 창을 닫은 뒤 후속 창과 잔존 여부를 재확인하도록 절차를 보완했습니다.
정리 완료와 실패를 반환 코드로 구분해, 후속 작업에서 결과를 판단할 기준을 마련했습니다.

재현 시연 · 정리 확인 후 진행하는 절차



비교 재현 영상 ↗

개선 방향 · 정리 완료 후 다음 작업 진행

오류 창을 닫은 뒤 일정 간격으로 재확인하고,
남은 창이 있으면 종료를 반복합니다.
정리 완료 확인 후 Clean/Rebuild하도록 제안했습니다.

구현 내용 · 반복 정리와 결과 반환

후속 창을 구분하고 반복 정리·안정 대기를 수행합니다.
같은 창이 남거나 대기 후 오류가 감지되면
실패 코드를 반환합니다.

검증 범위 · 오류 정리·결과 판별 코드

검증 대상은 오류 정리·결과 판별 코드입니다.
사진은 절차를 보여주는 재현 시연이며,
이번 재검증은 DLL 복구·촬영 재개를 포함하지 않습니다.

오류 창 정리 코드 ↗

작업 기여 | **사용자 작업:** 잔존 오류 재확인 절차 제안 | **Codex 작업:** 반복 정리·재검사·결과 반환 구현

검증: 순차 오류와 실패 조건을 실제로 확인

보완된 오류 창 정리 코드를 추가 수정 없이, 테스트 창으로 5개 조건을 각각 3회 실행했습니다.
종료 코드·잔존 오류·일반 창 유지 여부를 확인해, 오류 창 정리 성공과 잔존 오류 감지를 구분했습니다.

검증 범위 · 재현용 테스트 창 · 5개 조건 × 3회 · 2026.10.19

테스트 조건	기대 동작	확인한 결과	판정
오류 없는 정상 상태	창을 닫지 않고 정리 절차 종료	종료 코드 0 · 일반 창 1개 유지	**3/3 통과**
A 종료 후 B 표시	B까지 정리 후 성공 코드 반환	종료 코드 0 · 오류 잔존 0개	**3/3 통과**
안정 대기 중 B 표시	남은 B 감지 · 성공 판정 안 함	종료 코드 3 · B 잔존 1개 감지	**3/3 실패 감지[1]**
종료 요청을 거부하는 창	반복 중단 · 실패 코드 반환	종료 코드 3 · 같은 창 잔존 감지	**3/3 실패 감지[1]**
오류 창과 일반 창 동시 표시	오류만 정리 · 일반 창 유지	종료 코드 0 · 오류 0 / 일반 창 1	**3/3 통과**

[1] 잔존 오류를 감지해 실패 코드를 반환한 결과이며, 오류 창 정리 성공을 의미하지 않습니다.

확인한 한계 지연 오류는 감지 후 실패 종료하며 자동 재정리하지 않습니다. 실제 DLL 복구·재촬영은 미검증입니다.
별도 테스트는 완료 대기 한도 5초에서 실패하고 15초에서 통과했으며, 실패 원인은 미확정입니다.

검증 보고서 · 15회 실행 ↗

대기 한도 보완 기록 · 5초/15초 ↗

이번 재검증 | **사용자 작업:** 재검증 방향·조건 검토 | **Codex 작업:** 테스트 구성·실행·로그 수집

문제: FBX 임포트 구조 불일치로 캐릭터 형태 붕괴

캐릭터가 한곳에 뭉쳐 정상적인 형태로 표시되지 않았습니다.
임포트된 본 계층과 애니메이션 채널을 확인해, 기존 로더가 기대한 구조와의 차이를 추적했습니다.

캐릭터 형태 붕괴 · Pivot 보존 ON



Assimp 6.0.2 · 동일 조건 비교 2026.10 촬영

Assimp 옵션 전후 비교 영상 ↗

관찰한 현상 · 몸이 한곳에 뭉침

머리·몸·다리가 겹치며 캐릭터의 형태가
무너지고, 보행 자세가 제대로 표현되지 않았습니다.

구조 확인 · Pivot 보존으로 달라진 이름

로더가 찾는 본 이름

mixamorig:Spine

임포트된 채널 이름 · 실제 로그 발췌

mixamorig:Spine_$AssimpFbx$_Rotation

원인 · 기존 로더의 채널 매핑 실패

이름이 맞지 않는 채널이 본 인덱스 0에 연결돼,
여러 채널의 변환이 같은 본에 덮어써졌습니다.

보행 클립: 보조 노드 144개 / 미일치 채널 48개

임포트 구조·채널 매핑 로그 ↗

작업 기여 | **사용자 작업:** 당시 문제 인지·문제 제시 | **Codex 작업:** 이번 동일 조건 재현·구조 로그 수집

해결: Pivot 보존 설정을 로더의 처리 구조에 맞춤

기존 로더가 처리하는 본 계층과 채널 이름에 맞도록 Pivot 보존 옵션을 껐습니다.
모델과 애니메이션을 읽는 경로에 같은 설정을 적용했습니다.

캐릭터 형태 복원 · Pivot 보존 OFF



앞 페이지와 동일 프레임·배율 2026.10 촬영

Assimp 옵션 전후 비교 영상 ↗

변경 코드 · 동일한 임포트 설정 적용

ModelLoader의 Load / LoadAnimation에서
ReadFile 호출 전에 동일 옵션을 적용했습니다.

```
importer.SetPropertyBool(
    AI_CONFIG_IMPORT_FBX_PRESERVE_PIVOTS,
    false);
```

변경 의미 · 로더가 처리할 수 있는 구조

Pivot 변환을 보조 노드로 분리해 보존하는 대신,
기존 로더가 처리하는 본·채널 구조로 읽었습니다.
임포트 설정을 로더의 처리 조건에 맞춘 변경입니다.

변경 근거: 2026.07 · 커밋 7c061ee
최초 분석·코드 작성자는 기록으로 미확인.

당시 변경·확인 기록 ↗ 당시 실행 확인 원문 ↗

작업 기여 | **사용자 작업:** 당시 수정 후 Debug/Release 확인 | **Codex 작업:** 이번 옵션 전후 실행·계측

검증: 채널 매칭과 보행 정상화 확인

동일한 자산과 Assimp 버전에서 Pivot 보존 옵션을 비교했습니다.
5개 클립의 임포트 구조를 검사하고, 보행 클립의 실제 재생 결과를 확인했습니다.

옵션 변경 전후 · 구조 검사와 실제 보행 재생

검증 항목	확인한 결과 · ON → OFF	검증 범위
임포트 구조와 이름 매칭	보조 노드 144 → 0개 / 미일치 채널 48 → 0개	5개 클립 각각
미등록 채널 처리	본 인덱스 0으로 처리되는 경로 240 → 0회	5개 클립 합계
실제 애니메이션 재생	캐릭터 형태와 보행 자세 복원	보행 클립 240프레임

Pivot 보존 ON



Pivot 보존 OFF



Assimp 옵션 전후 비교 영상 ↗

확인 범위와 한계

구조 검사는 5개 클립, 실제 재생 비교는 보행 클립에 한정했습니다.
다른 클립 전체 재생·장시간 실행·임의 자산은 미검증입니다.

비교 조건 · 2026.10 촬영

양쪽에 Codex의 초기 자세 보완을 동일 적용하고 Pivot 옵션만 변경했습니다.
추가 보완의 코드·검사 결과는 상세 보고서에 별도로 기록했습니다.

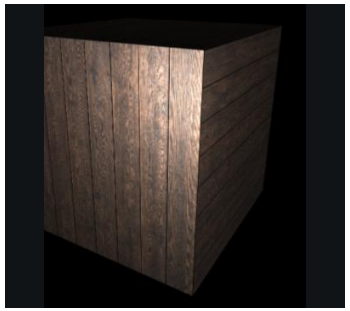
검증 보고서 · 조건·로그·한계 ↗

작업 기여 | **사용자 작업:** 당시 수정 결과 직접 실행 확인 | **Codex 작업:** 이번 동일 조건 비교·로그·영상 수집

대표 그래픽스 학습·구현 범위

교재·강의 기반 예제를 학습하며 구현·실행한 결과를 정리한 아카이브입니다.
학습 코드와 개인 확장, 이후 Codex를 활용한 조사·문서화·실행 자료 정리를 구분합니다.

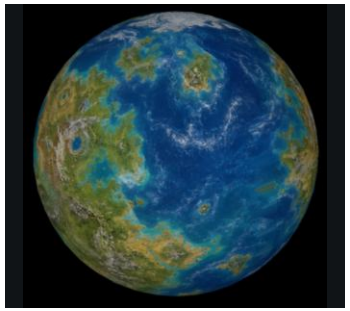
DirectX11 렌더링 기초



리소스·셰이더 연결
텍스처 기반 화면 출력

예제 문서·결과 이미지 ↗

절차적 메시 생성



정점·인덱스 구성
법선·UV 생성

예제 문서·결과 이미지 ↗

재질·조명



PBR·IBL
그림자 표현

예제 문서·결과 이미지 ↗

사용자 상호작용



화면 좌표 기반 피킹
쿼터니언 회전·이동

예제 문서·결과 이미지 ↗

스켈레탈 애니메이션



본 계층 변환·스키닝
애니메이션 클립 재생

예제 문서·결과 이미지 ↗

GPU 연산



Compute Shader
속도·밀도 갱신

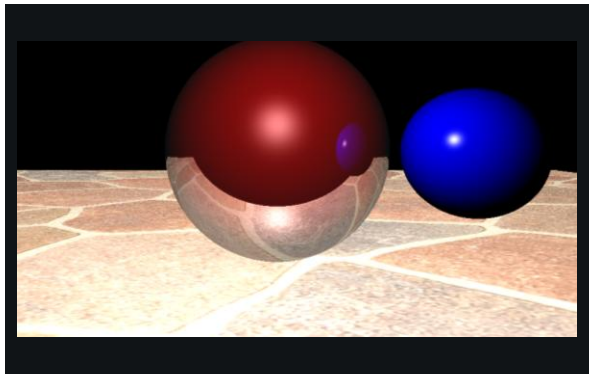
예제 문서·결과 이미지 ↗

공통 범위 | **사용자 작업:** 학습·정리 기준 제시·결과 검토 | **Codex 작업:** 코드·학습 기록 조사, 문서화·실행 자료 정리

CPU Ray Tracing의 반사·굴절

광선과 물체의 교차점을 찾고, 반사·굴절 방향의 광선을 재귀적으로 추적했습니다.
CPU에서 계산한 픽셀 데이터를 DirectX11 텍스처에 복사해 화면에 표시했습니다.

반사 예제



굴절 예제



서로 다른 예제 장면의 결과이며, 동일 장면의 적용 전후 비교는 아닙니다.

핵심 원리 · 반사·굴절 광선

교차점의 법선과 굴절률로 후속 광선의 방향을 구하고,
그 광선이 도달한 표면의 색을 계산합니다.

구현 방식 · 최근접 교차점과 재귀 추적

최근접 교차점을 선택하고 재귀 깊이를 제한합니다.
시작점에 작은 오프셋을 더해 자기 교차를 줄입니다.

결과 해석 · 반사상과 굴절 표현

각 예제에서 반사상과 굴절된 배경 표현을 확인합니다.
두 이미지는 같은 장면의 적용 전후 비교가 아닙니다.

반사 광선의 재귀 추적 · C++

원문 발췌

반사 광선을 추적한 색에 반사율을 곱해 누적합니다.

```
color += traceRay(reflectedRay, depth - 1)
    * hit.obj->reflection;
```

굴절 광선의 방향 계산 · C++

원문 발췌

광선 방향·법선·굴절률 비로 굴절 방향을 구합니다.

```
const vec3 refractedDir =
    glm::refract(ray.dir, normal, eta);
```

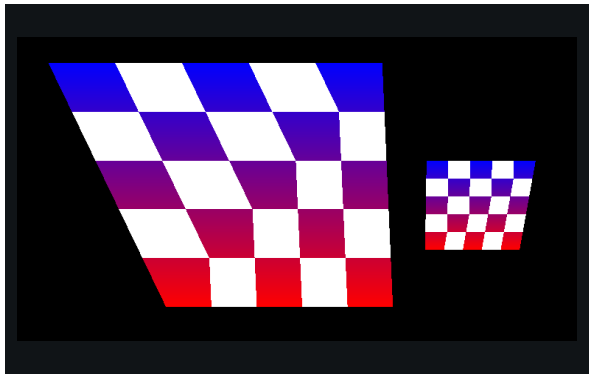
반사 문서·결과 이미지 ↗ 굴절 문서·결과 이미지 ↗ 반사 구현 코드 ↗ 굴절 구현 코드 ↗

학습·구현 범위 | 교차점 판정, 반사·굴절 광선의 재귀 추적과 픽셀 버퍼 출력

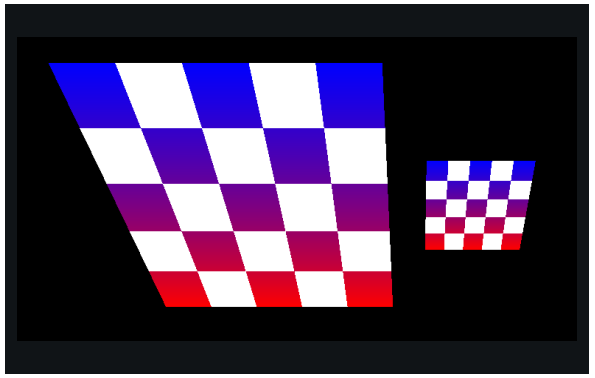
원근 투영과 UV 보간 보정

원근 투영 이후의 화면 공간 가중치에 깊이를 반영해 UV를 보간했습니다.
같은 위치·투영 조건에서 보정 전후 체크 패턴의 변화를 비교한 기존 자료입니다.

UV 보정 전



UV 보정 후



화면 거리 1.000 · 왼쪽 Z 0.309 · 오른쪽 Z 2.490 고정

핵심 원리 · 원근 투영 뒤의 보간

화면 공간의 보간 비율은 원래 공간의 비율과 다릅니다.
깊이를 반영해야 텍스처의 원근감을 유지할 수 있습니다.

구현 방식 · 깊이로 나눈 가중치

화면 가중치를 눈 기준 깊이로 나누고 다시 정규화합니다.
이 예제는 CPU에서 UV를 보간하고 픽셀을 계산합니다.

결과 해석 · 체크 패턴의 변화

같은 위치·투영 조건에서 보정 전후의 패턴을 비교합니다.
DirectX11은 CPU가 계산한 픽셀 버퍼를 화면에 표시합니다.

원근 보정 가중치 계산 · C++

원문 발췌

화면 가중치를 깊이로 나눈 뒤 전체 합으로 정규화합니다. 다른 정점도 같은 방식으로 처리합니다.

```
const float ratio0 = w0 / z0;
const float ratioTotal = ratio0 + ratio1 + ratio2;
w0 = ratio0 / ratioTotal;
```

예제 문서·결과 이미지 ↗

깊이 보정 코드 ↗

학습·구현 범위 | 깊이를 반영한 가중치 정규화와 원근 보정 UV 보간

DirectX11의 데이터·셰이더 연결

정점·인덱스·텍스처와 상수 버퍼를 연결해, 형상과 조명을 화면에 표시했습니다.
예제의 실제 바인딩 코드와 DrawIndexed 호출을 기준으로 데이터 흐름을 정리했습니다.

텍스처·조명 적용 결과



Texturing LightingSelf 실행 결과

01 입력 데이터
정점·인덱스·텍스처·상수

↓

02 셰이더 처리
Vertex Shader / Pixel Shader

↓

03 렌더 타깃
색상 출력 / 깊이 검사

핵심 원리 · 입력과 셰이더의 연결

정점·인덱스는 형상을, 텍스처·상수 버퍼는
변환과 재질·조명 계산에 필요한 값을 제공합니다.

구현 방식 · 바인딩과 DrawIndexed

정점·인덱스 버퍼와 셰이더 입력을 연결합니다.
렌더 타깃·깊이 버퍼를 설정하고 DrawIndexed로 출력합니다.

결과 해석 · 형상·재질·조명의 결합

실행 화면에서 텍스처와 조명이 적용된 형상을 확인합니다.
개별 입력의 효과를 분리한 비교 실험은 아닙니다.

원문 발췌

텍스처·인덱스 버퍼 연결과 그리기 · C++

픽셀 셰이더에 텍스처를, 입력 단계에 인덱스 버퍼를 연결한 뒤 삼각형을 그립니다.

```
m_context->PSSetShaderResources(0, 1, pixelResources_0);
m_context->IASetIndexBuffer(m_indexBuffer.Get(), DXGI_FORMAT_R16_UINT, 0);
m_context->DrawIndexed(m_indexCount, 0, 0);
```

예제 문서·결과 이미지 ↗

리소스 연결·Draw 호출 코드 ↗

학습·구현 범위 | 정점·인덱스·텍스처·상수 버퍼 연결과 DrawIndexed 호출

Normal Mapping의 법선 변환

Normal Map의 방향을 표면의 TBN 기저로 변환해 조명 계산에 사용했습니다.
정점 위치를 바꾸지 않고, 표면의 밝기 변화로 요철을 표현하는 예제입니다.

Normal Mapping · Off



Normal Mapping · On



같은 영역·배율로 발췌 · 조명 반응의 변화 비교

핵심 원리 · 법선으로 표현하는 요철

Normal Map의 방향을 조명 계산에 사용해
정점 위치를 바꾸지 않고 표면의 요철을 표현합니다.

구현 방식 · 방향 복원과 TBN 변환

RGB 값을 -1~1 범위로 복원하고 TBN 기저로 변환합니다.
월드 공간의 법선을 정규화해 조명 계산에 사용합니다.

결과 해석 · 조명 반응과 형상의 차이

같은 영역에서 표면의 밝기·무늬 변화를 비교합니다.
형상·외곽선이나 별도의 자기 그림자를 만들지는 않습니다.

법선 복원과 TBN 변환 · HLSL

원문 발췌

텍스처의 방향 값을 복원하고 월드 공간으로 변환한 뒤 정규화합니다.

```
normalTex = 2.0 * normalTex - 1.0;
float3x3 TBN = float3x3(T, B, N);
normalWorld = normalize(mul(normalTex, TBN));
```

예제 문서·결과 이미지 ↗

법선·TBN 변환 코드 ↗

학습·구현 범위 | 법선 맵 방향 복원과 TBN 변환을 통한 조명 계산

Height Mapping의 정점 변위

Vertex Shader에서 높이 텍스처를 읽고 법선 방향으로 정점을 이동했습니다.
Normal Mapping과 달리, 실제 표면 위치와 외곽선이 달라지는 예제입니다.

Height Mapping · Off



Height Mapping · On



같은 영역·배율로 발췌 · Height Scale 0.1 적용

핵심 원리 · 높이 값에 따른 정점 변위

높이 텍스처를 이용해 정점을 법선 방향으로 이동합니다.
Normal Mapping과 달리 실제 표면 위치가 바뀝니다.

구현 방식 · 높이 복원과 변위 적용

Vertex Shader에서 높이를 -1~1로 복원합니다.
법선·높이·Height Scale을 곱해 정점 위치에 더합니다.

결과 해석 · 외곽선과 정점 밀도

Scale 0.1 적용 전후의 표면·외곽선을 비교합니다.
정점 밀도에 영향을 받으며, 변위 후 법선은 갱신하지 않습니다.

원문 발췌

높이 값에 따른 정점 이동 · HLSL

높이를 -1~1로 복원하고 법선 방향의 변위를 정점 위치에 더합니다.

```
height = 2.0 * height - 1.0;
pos += float4(output.normalWorld * height * heightScale, 0.0);
```

예제 문서·결과 이미지 ↗

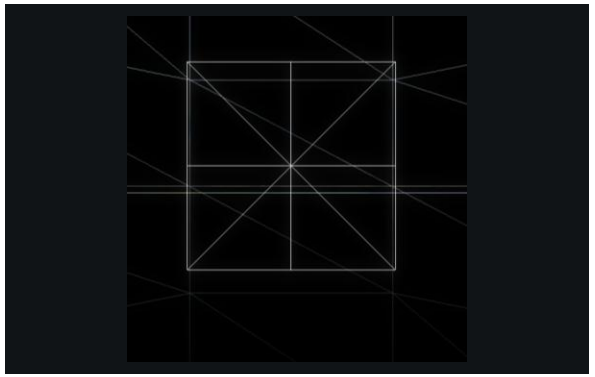
정점 변위 코드 ↗

학습·구현 범위 | 높이 텍스처를 이용한 법선 방향 정점 변위

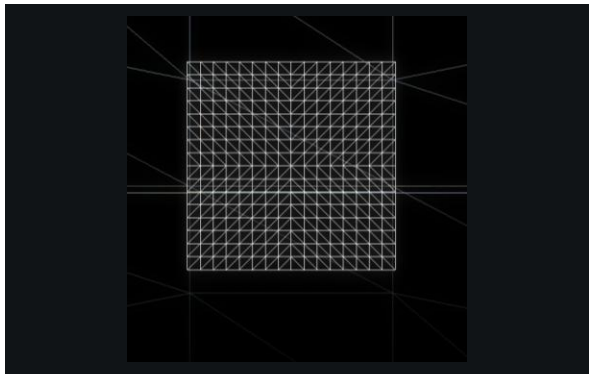
Tessellation의 분할 밀도 제어

네 개의 Control Point로 구성한 패치를 Hull Shader의 분할 계수에 따라 세분화했습니다.
같은 카메라에서 Manual 계수만 바꾼 기존 실행 결과로 삼각형 밀도를 비교했습니다.

Manual · Factor 2



Manual · Factor 16



동일 카메라 · Manual 분할 계수 2 / 16

핵심 원리 · 패치의 세분화

분할 계수를 높이면 패치의 정점·삼각형이 늘어납니다.
Hull·Domain Shader가 분할과 정점 위치 계산을 나눕니다.

구현 방식 · 분할 계수와 정점 위치

Hull Shader가 외곽·내부 분할 계수를 설정하고,
Domain Shader는 패치 내부 좌표로 정점 위치를 구합니다.

결과 해석 · Factor 2와 16의 차이

같은 카메라에서 Manual 계수에 따른 삼각형 밀도를 봅니다.
거리 적응형 분할은 이 비교에 포함하지 않습니다.

패치 분할 계수 전달 · HLSL

원문 발췌

외곽·내부 분할 계수를 패치 상수에 기록합니다. 정점 위치 계산은 Domain Shader에서 수행합니다.

```
pt.edges[0] = edges.x;
pt.inside[0] = inside.x;
pt.inside[1] = inside.y;
```

예제 문서·결과 이미지 ↗

분할 계수 코드 ↗

학습·구현 범위 | Hull Shader 분할 계수와 Domain Shader의 정점 위치 계산

PBR·IBL의 재질 입력과 조명 계산

재질 맵과 환경광 텍스처를 셰이더 입력으로 연결한 PBR·IBL 학습 예제입니다.
각 입력의 역할을 실제 코드와 여러 맵을 함께 적용한 실행 결과로 정리했습니다.

PBR·IBL 종합 적용 결과



여러 재질 맵과 Height Scale 0.015를 함께 적용한 결과

재질 입력
Albedo·Metallic
Roughness
Normal·AO

환경광 입력
Irradiance
Specular 맵
LUT

조명 결과
표면 방향·시선·거칠기 반영

핵심 원리 · 재질과 환경광

재질·표면 방향·시선 방향에 따라 조명 반응을 구합니다.
환경 맵은 주변에서 들어오는 빛의 입력으로 사용합니다.

구현 방식 · 재질 맵과 IBL 샘플링

재질 맵과 Irradiance·Specular 환경 맵, LUT를 연결합니다.
Roughness에 따라 반사 환경 맵의 Mip Level을 선택합니다.

결과 해석 · 여러 입력의 종합 결과

재질·환경광 맵과 Height Scale 0.015를 함께 적용했습니다.
각 맵의 효과를 하나씩 분리한 비교는 아닙니다.

확산·반사 환경광 샘플링 · HLSL

원문 발췌

법선 방향의 확산광과 반사 방향의 환경광을 읽고, 거칠기로 반사 샘플의 Mip Level을 정합니다.

```
float3 irradiance = irradianceIBLTex.Sample(linearSampler, normalWorld).rgb;
float3 specularIrradiance = specularIBLTex.SampleLevel(linearSampler,
    reflect(-pixelToEye, normalWorld), roughness * 10.0).rgb;
```

예제 문서·결과 이미지 ↗

IBL 샘플링 코드 ↗

학습·구현 범위 | 재질 맵 입력과 환경 맵 샘플링을 사용하는 PBR·IBL 조명 계산

HDR·Bloom의 후처리 흐름

부동소수점 장면 버퍼에서 Bloom 이미지를 만들고, 원본과 합성했습니다.
현재 활성 코드의 가중 혼합·노출·감마 처리를 기준으로 후처리 흐름을 설명합니다.

Bloom Strength · 0



Bloom Strength · 0.7



같은 영역·배율로 발췌 · 원본과 Bloom의 가중 혼합

핵심 원리 · 빛 번짐과 합성

해상도를 줄인 이미지를 다시 확대해 밝은 영역을 퍼뜨립니다.
이를 원본 장면과 혼합해 Bloom 효과를 만듭니다.

구현 방식 · 가중 혼합과 표시 변환

원본과 Bloom을 (1−strength), strength 비율로 섞습니다.
이후 노출·0~1 제한·감마 처리를 적용합니다.

결과 해석 · 강도에 따른 변화

같은 영역에서 Strength 0과 0.7의 빛 번짐을 비교합니다.
현재 혼합식은 강도가 커질수록 원본의 비중도 줄어듭니다.

원본·Bloom 혼합과 표시 변환 · HLSL

원문 발췌

두 이미지의 비중을 strength로 조절하고, 선택된 LinearToneMapping 경로로 결과를 변환합니다.

```
float3 combined = (1.0 - strength) * color0 + strength * color1;
combined = LinearToneMapping(combined);
```

예제 문서·결과 이미지 ↗

Bloom 합성·톤 매핑 코드 ↗

학습·구현 범위 | Bloom 이미지 생성·가중 합성과 노출·감마 처리

화면 입력과 3D 선택·조작

화면 좌표를 선택 광선으로 바꾸고, 구 표면의 드래그 입력으로 회전을 적용했습니다.
Virtual Trackball 예제의 선택 지점과 회전 누적 과정을 기존 자료로 설명합니다.

드래그 전



드래그 중



기존 시연의 초기·드래그 상태 · 같은 영역·배율로 발췌 · 노란 점은 선택 지점

핵심 원리 · 화면 입력을 3D 회전으로

화면 좌표로 선택 광선을 만들고 구 표면의 지점을 구합니다.
직전·현재 선택 지점의 방향 차이를 회전량으로 사용합니다.

구현 방식 · 선택 판정과 회전 누적

교차를 검사하고 두 벡터 사이의 쿼터니언을 구합니다.
모델에 회전을 합성하고 현재 벡터를 다음 기준으로 둡니다.

결과 해석 · 드래그에 따른 자세 변화

드래그 전·중 상태에서 모델의 회전을 확인합니다.
노란 점은 선택 지점이며, 연속 동작은 조작 영상에 있습니다.

선택 판정과 회전량 계산 · C++

원문 발췌

선택 광선의 교차를 검사하고 두 방향 사이 회전을 구합니다.

```
m_selected = curRay.Intersects(m_mainBoundingSphere, dist);
q = Quaternion::FromToRotation(prevVector, currentVector);
```

모델 행렬에 회전 합성 · C++

원문 발췌

구한 쿼터니언을 회전 행렬로 바꿔 기존 변환에 합성합니다.

```
m_mainSphere.UpdateModelWorld(m_mainSphere.m_modelWorldRow *
    Matrix::CreateFromQuaternion(q) *
    Matrix::CreateTranslation(translation));
```

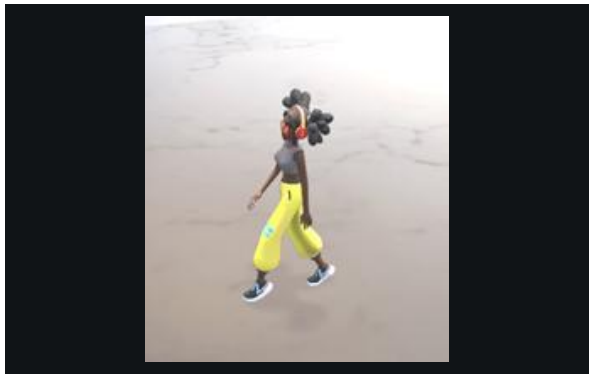
예제 문서·결과 이미지 ↗
실행 영상 ↗
선택·회전 처리 코드 ↗

학습·구현 범위 | 화면 좌표 기반 피킹과 쿼터니언 회전 누적

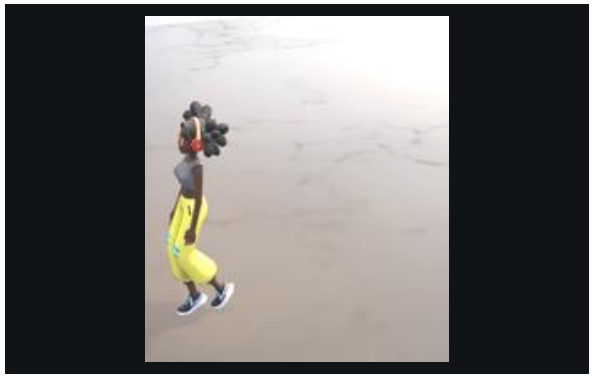
스켈레탈 애니메이션의 변환 흐름

클립의 본 변환을 계층에 따라 갱신하고, GPU에서 가중합으로 정점을 변형했습니다.
기존 애니메이션 시연의 두 시점을 같은 배율로 발췌했습니다.

기존 시연 · 0.800초



기존 시연 · 4.300초



동일 클립 재생의 0.800초·4.300초 장면 · 같은 배율로 발췌

핵심 원리 · 본 변환의 가중합

각 정점에 연결된 본의 변환을 가중치만큼 합산합니다.
여러 본의 영향으로 메시가 애니메이션 자세로 변형됩니다.

구현 방식 · CPU 갱신과 GPU 스키닝

클립·프레임의 본 계층 변환을 계산해 GPU로 전달합니다.
셰이더는 본 인덱스·가중치로 위치·법선·접선을 변환합니다.

결과 해석 · 동일 클립의 두 시점

0.800초와 4.300초의 자세 차이로 재생 결과를 확인합니다.
FBX 호환 문제의 원인과 해결은 7~9페이지에서 다룹니다.

본 변환에 따른 정점 스키닝 · HLSL

원문 발췌

각 본으로 변환한 위치·법선·접선을 해당 가중치만큼 더합니다. 아래는 반복문 내부의 누적 연산입니다.

```
posModel += weights[i] * mul(float4(input.posModel, 1.0f), boneTransforms[indices[i]]).xyz;
normalModel += weights[i] * mul(input.normalModel, (float3x3)boneTransforms[indices[i]]);
tangentModel += weights[i] * mul(input.tangentModel, (float3x3)boneTransforms[indices[i]]);
```

예제 문서·결과 이미지 ↗ 실행 영상 ↗ 정점 스키닝 코드 ↗

학습·구현 범위 | 본 계층 변환 갱신과 본 가중치를 이용한 GPU 스키닝

GPU Blur의 구현 경로와 리소스 교대

A/B 리소스를 읽기·쓰기 대상으로 교대하며 Blur의 X/Y 방향 계산을 수행했습니다.
현재 활성 Compute Shader 경로의 데이터 전달과 Dispatch 순서를 설명합니다.

기존 Blur 실행 결과



현재 활성 Compute Shader 경로의 기존 실행 결과

A · 입력 리소스

↓ ① X 방향 계산 · A → B

B · X 방향 계산 결과

↓ ② Y 방향 계산 · B → A

A · Y 방향 계산 결과

핵심 원리 · 입력과 출력의 교대

한 단계의 출력이 다음 단계의 입력이 되도록
A/B 리소스를 번갈아 읽고 쓰는 방식입니다.

구현 방식 · SRV·UAV와 X/Y 패스

X 방향은 A 읽기·B 쓰기, Y 방향은 B 읽기·A 쓰기입니다.
패스 사이에 바인딩을 해제하고 각 커널을 Dispatch합니다.

결과 해석 · Blur 결과와 검증 범위

활성 Compute Shader 경로로 생성한 Blur 결과입니다.
바인딩 해제는 완료 대기나 성능 측정을 뜻하지 않습니다.

① X 패스의 입력·출력 연결 · C++

원문 발췌

A를 읽는 SRV와 B에 쓰는 UAV를 연결합니다.

```
m_context->CSSetShaderResources(0, 1, m_srvA.GetAddressOf());
m_context->CSSetUnorderedAccessViews(
    0, 1, m_uavB.GetAddressOf(), NULL);
```

예제 문서·결과 이미지 ↗

② Y 패스의 입력·출력 연결 · C++

원문 발췌

앞 단계 결과 B를 읽고 A에 쓰도록 역할을 바꿉니다.

```
m_context->CSSetShaderResources(0, 1, m_srvB.GetAddressOf());
m_context->CSSetUnorderedAccessViews(
    0, 1, m_uavA.GetAddressOf(), NULL);
```

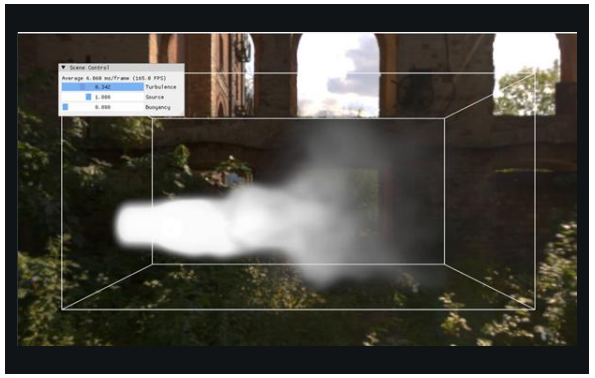
A/B 리소스 교대 코드 ↗

학습·구현 범위 | SRV/UAV 바인딩 해제와 A/B 리소스 교대를 통한 X/Y Blur

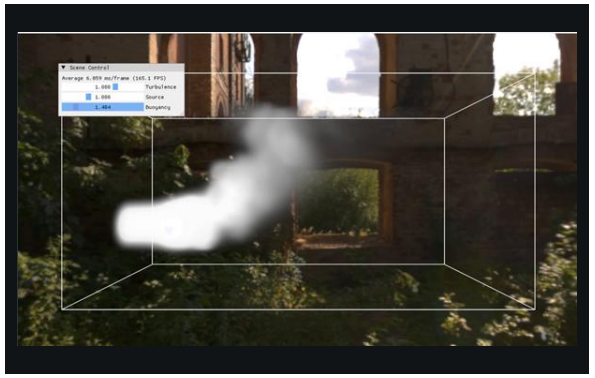
GPU 연기의 시뮬레이션과 볼륨 렌더링

Compute Shader로 3차원 속도·밀도 필드를 갱신하고, 밀도 텍스처를 렌더링했습니다.
기존 영상의 연기 형태 변화와 속도를 이용한 밀도 이송 연산을 정리했습니다.

4.025초



22.808초



동일 시연의 4.025초·22.808초 장면 · 같은 영역·배율로 발췌

핵심 원리 · 속도에 따른 밀도 이동

3차원 속도·밀도 필드를 갱신해 연기의 이동을 표현합니다.
속도를 거슬러 찾은 이전 위치의 밀도를 현재 셀에 반영합니다.

구현 방식 · 필드 갱신과 볼륨 표시

Source 주입·발산·압력 계산·속도 보정 뒤 밀도를 이송합니다.
Ray Marching으로 밀도를 누적해 화면에 표시합니다.

결과 해석 · 시간에 따른 연기 형태

동일 시연의 4.025초·22.808초에서 연기 형태를 비교합니다.
시각 결과를 보여주며, 물리 정확도를 측정한 자료는 아닙니다.

속도를 거슬러 밀도 샘플링 · HLSL

원문 발췌

속도·시간 간격으로 이전 위치를 역추적하고, 그 위치의 밀도를 읽어 현재 값을 갱신합니다.

```
float3 vel = velocityTemp[dtID.xyz].xyz * dx;
float3 uvwBack = uvw - (vel * dt * upScale);
density[dtID.xyz] = densityTemp.SampleLevel(linearWrapSS, uvwBack, 0.0) * 0.999;
```

예제 문서·결과 이미지 ↗ 실행 영상 ↗ 시뮬레이션 단계 코드 ↗ 밀도 이송 셰이더 ↗

학습·구현 범위 속도·밀도 필드 갱신과 Ray Marching 기반 볼륨 렌더링

잔디 인스턴싱과 셰이더 변형

하나의 잔디 형상과 개별 배치 데이터를 분리해 인스턴스 렌더링을 구성했습니다.
Vertex Shader에서 위치·시간·바람 세기로 잔디의 움직임을 표현했습니다.

기존 시연 · 1.190초



기존 시연 · 6.743초



동일 시연의 시간에 따른 잔디 방향 변화

핵심 원리 · 형상 공유와 인스턴스 출력

잔디 형상은 공유하고 위치·회전·크기는 개별 데이터로 둡니다.
인스턴스 데이터로 같은 형상을 여러 위치에 출력합니다.

구현 방식 · 배치와 바람 변형

두 버퍼를 연결하고 DrawIndexedInstanced를 호출합니다.
셰이더가 위치·시간·바람에 따라 정점과 법선을 회전시킵니다.

결과 해석 · 잔디 움직임과 구현 범위

100,000개를 구성하고 시간에 따른 잔디 방향 변화를 봅니다.
충돌·GPU Culling·LOD는 구현 범위에 포함하지 않습니다.

인스턴스 단위로 형상 출력 · C++

원문 발췌

같은 인덱스 형상을 인스턴스 수만큼 출력합니다.

```
context->DrawIndexedInstanced(
    m_indexCount, m_instanceCount, 0, 0, 0);
```

바람 회전을 정점에 적용 · HLSL

원문 발췌

바람 회전을 행렬로 만들고 월드 정점 위치에 적용합니다.

```
float4 q = rotate_angle_axis(input.windStrength, axis);
mWind = quaternion_to_matrix(q);
output.posWorld = mul(float4(output.posWorld, 1.0f), mWind).xyz;
```

예제 문서·결과 이미지 ↗ 실행 영상 ↗ 인스턴스 출력 코드 ↗ 바람 변형 셰이더 ↗

학습·구현 범위 | 인스턴스 데이터 기반 반복 출력과 시간·바람에 따른 정점 변형